PYTHON PROGRAMMING

10 Comprehensive Problem Statements

Covering All Core Python Concepts

Variables • Datatypes • Literals • Base Conversions • Type Conversions • Operators • Control Flow • Collections • Strings • Functions • OOP • File Handling • Iterators • Generators

Prepared for Advanced Python Learning

May 2026

Concepts Index

Every problem statement in this document exercises ALL of the following Python concepts:

Variables, Datatypes, Literals Arithmetic, Bitwise Operators	Type Conversions & Base Conversions Logical & Comparison Operators
if / elif / else	match-case (Python 3.10+)
Lists + List Comprehension Sets & frozensets for / while loops	Tuples (immutable sequences) Dictionaries (all methods) break / continue / pass
Strings (all methods)	Functions (all argument types)
Exception Handling (try/except) Iterators (__iter__/__next__) OOP: Inheritance	File Handling (read/write/append) Generators (yield) OOP: Polymorphism
OOP: Operator Overloading	OOP: Abstraction (ABC)

Problem Index

#	Problem Title	Domain	Difficulty
1	Smart Library Management System	Library / Inventory	Intermediate
2	Student Grade Analytics Portal	Education / Analytics	Beginner–Intermediate
3	E-Commerce Order Processing Engine	E-Commerce / Finance	Intermediate–Advanced
4	Hospital Patient Record System	Healthcare / Data Management	Intermediate
5	Banking & Financial Transactions Simulator	Finance / Banking	Intermediate–Advanced
6	Smart Inventory & Supply Chain Manager	Logistics / Inventory	Intermediate
7	Social Media Analytics Dashboard	Social Media / Analytics	Intermediate
8	Airline Flight Reservation System	Travel / Reservation	Advanced
9	Smart Home Automation Controller	IoT / Home Automation	Intermediate
10	Text-Based RPG Game Engine	Game Development / Entertainment	Advanced

Problem Statements

Problem 1: Smart Library Management System

Difficulty	Intermediate	Domain	Library / Inventory

Objective

Build a fully functional library management system that models books, members, and lending operations using a modular, object-oriented Python program.

Problem Description

You are tasked with creating a command-line library management system. The system must handle book catalogues, member registrations, borrowing/returning operations, and fine calculation. Every layer of the program must demonstrate the breadth of Python concepts listed below.

Tasks to Implement

1. Define a Book class with attributes: title, author, isbn (int), price (float), available (bool). Use __init__, __str__, and __repr__.
2. Use a list to store all Book objects. Demonstrate list comprehension to filter available books by genre stored in a tuple.
3. Implement a Dictionary to map member_id (int) to member details (nested dict). Use all standard dict methods: get(), items(), keys(), values(), update(), pop().
4. Create a set of banned_isbns and perform union, intersection, and difference operations with returned books.
5. Write a generator function book_catalogue_generator() that lazily yields books one at a time from the list.
6. Implement a custom Iterator class LibraryIterator with __iter__ and __next__ to walk through the catalogue.
7. Demonstrate all arithmetic operators to compute late fees: fee = (days_late * daily_rate) ** penalty_multiplier % max_cap.
8. Use bitwise operators to encode book status flags (available=1, reserved=2, damaged=4) in a single integer.
9. Apply logical and comparison operators in a multi-condition eligibility check for borrowing privileges.
10. Use if / elif / else and match-case (Python 3.10+) to route member requests to the correct handler.
11. Demonstrate base conversions: store ISBNs in hex, display in octal, accept binary input from librarians.
12. Implement type conversions: parse string inputs to int/float, convert booleans to int for flag arithmetic.
13. Write string methods: strip(), split(), join(), upper(), lower(), replace(), find(), startswith(), endswith(), format(), encode(), count() on book titles and author names.
14. Use for loops with enumerate and zip to pair books with shelf locations stored in a tuple.
15. Use while, break, continue, and pass in an interactive menu loop.
16. Apply try/except (with multiple except clauses, else, finally) for ISBN validation, file I/O, and type conversion errors.
17. Save and load the catalogue using file handling (open, read, write, with statement) in both text and binary mode.
18. Implement a base class LibraryEntity and subclasses Book, EBook, AudioBook showing inheritance and method overriding (polymorphism).
19. Define abstract methods via ABC for checkout() and return_item().
20. Overload the + operator in a BookCollection class to merge two collections, and overload __eq__ to compare ISBNs.
21. Write modular functions using positional args, keyword args, *args, **kwargs, and keyword-only args (after *) for search and filter utilities.

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass

✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions ✔ Arithmetic Operators	✔ Functions (all arg types) ✔ Exception Handling
✔ Bitwise Operators ✔ Logical & Comparison Operators	✔ File Handling ✔ Iterators
✔ if / elif / else ✔ match-case	✔ Generators ✔ OOP: Inheritance
✔ Lists + List Comprehension ✔ Tuples	✔ OOP: Polymorphism ✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
class Book(LibraryEntity):  # inheritance
fee = (days_late * 2.5) ** 1.2 % 500   # arithmetic
status = status | 0b0010               # bitwise OR to set 'reserved'
available_books = [b for b in catalogue if b.available]  # list comp
```

Expected Input / Output

Input: menu-driven CLI (1=Add Book, 2=Borrow, 3=Return, 4=Search, 5=Report). Output: formatted console reports + catalogue.txt file.

Problem 2: Student Grade Analytics Portal

Difficulty	Beginner–Intermediate	Domain	Education / Analytics

Objective

Develop a grade analytics portal that ingests student records, computes statistics, and exports formatted reports.

Problem Description

You will design a portal that reads student data from a CSV-like text file, processes marks across multiple subjects, applies grade boundaries, and generates a formatted report. The system must be modular and exception-safe.

Tasks to Implement

22. Represent each student as a dictionary: {id: int, name: str, marks: list[float], grade: str}. Use all dict operations.
23. Store the full student roster in a list. Use list methods: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear().
24. Use a tuple to hold the fixed grade boundaries (40, 55, 70, 85) and demonstrate tuple immutability vs list.
25. Build a set of unique subject names and perform set operations to detect missing or duplicate subjects.
26. Implement list comprehension to extract roll numbers of students who failed any subject.
27. Write a generator pass_fail_stream() that yields (name, status) tuples without loading all records into memory.
28. Create a custom iterator StudentRankIterator that yields students in descending order of aggregate.
29. Use all arithmetic operators: compute percentage, GPA (on a 10-point scale), and rank.

30. Demonstrate bitwise ops: encode pass(1)/fail(0) flags per subject in a bitmask for a 6-subject course.
31. Apply logical/comparison operators in eligibility checks for scholarships and supplementary exams.
32. Use if/elif/else to assign letter grades; use match-case to map grades to GPA points.
33. Demonstrate all base conversions: roll number in hex on ID cards, binary bitmask display, octal room codes.
34. Use all string methods on student names and subject strings: capitalize(), title(), zfill(), ljust(), rjust(), center(), format(), f-strings.
35. Use for with enumerate to number rank list; nested for to build a subject-wise marks matrix.
36. Use while loop for re-entry on invalid input; break on sentinel value; continue to skip blank lines; pass as placeholder.
37. Type-convert string marks from file to float; bool to int for bitmask; int to str for report formatting.
38. Wrap file reading, type conversion, and division (for average) in try/except/else/finally blocks.
39. Read student data from students.txt; write the final rank list to report.txt using context manager.
40. Design a Person base class, Student and Faculty subclasses overriding a display_info() method (polymorphism).
41. Abstract base class Evaluatable with abstract methods evaluate() and generate_report().
42. Overload + to merge two class sections (StudentGroup + StudentGroup) and __lt__ for sorting by GPA.
43. Functions: calculate_gpa(*marks), filter_students(students, *, min_gpa), search(students, name='', **filters).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions ✔ Arithmetic Operators ✔ Bitwise Operators	✔ Functions (all arg types) ✔ Exception Handling ✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else ✔ match-case	✔ Generators ✔ OOP: Inheritance
✔ Lists + List Comprehension ✔ Tuples	✔ OOP: Polymorphism ✔ OOP: Operator Overloading
✔ Sets ✔ Dictionaries	✔ OOP: Abstraction

Implementation Hints

```
gpa = round(sum(marks) / len(marks) / 10, 2)  # arithmetic + type conv
bitmask = 0b000000; bitmask |= (1 << i) if m>=40 else bitmask  # bitwise
roll_hex = hex(student['id']).upper()  # base conversion
name.center(20, '-').title()  # string methods
```

Expected Input / Output

Input: students.txt (CSV). Output: console rank table + report.txt with toppers highlighted.

Problem 3: E-Commerce Order Processing Engine

Difficulty	Intermediate–Advanced	Domain	E-Commerce / Finance

Objective

Simulate an end-to-end order processing pipeline for an online store, from cart management to invoice generation.

Problem Description

Build an order processing engine that manages products, carts, discounts, tax calculations, and invoice generation. The engine must handle concurrent-style operations, error recovery, and persistent storage.

Tasks to Implement

44. Model Product as a dataclass-style class with: product_id (int), name (str), price (float), stock (int), category (str).
45. Maintain an inventory as a list of Product objects. Use all list operations and list comprehension to filter by category, price range, and availability.
46. Store active carts as a dictionary: {user_id: [(product, qty)]}. Use get(), setdefault(), items() extensively.
47. Use a frozenset for immutable coupon codes and a set for used_coupons to prevent reuse.
48. Store tax slabs as a tuple of tuples: ((0,500,0.05),(500,2000,0.12),(2000,float('inf'),0.18)).
49. Write a generator invoice_line_generator(cart) that yields formatted invoice lines lazily.
50. Implement an OrderIterator class that iterates over all pending orders in chronological order.
51. Arithmetic: compute subtotal, discount amount (%), tax using slabs, grand total, change returned.
52. Bitwise: encode order status flags (placed=1, confirmed=2, shipped=4, delivered=8, cancelled=16).
53. Logical/comparison: validate coupon applicability, check stock sufficiency, verify payment amount.
54. if/elif/else for payment gateway routing; match-case for order status transitions.
55. Base conversions: order IDs in hex, transaction reference in binary, warehouse codes in octal.
56. Type conversions: parse price strings with currency symbols; convert bool flags to int for bitmask.
57. String methods: format invoice lines with ljust/rjust; parse product CSV with split/strip; build SKU codes with upper/replace/zfill.
58. for loop with zip to pair items and quantities; while loop for retry on payment failure; break/continue/pass in validation loops.
59. Exception handling: custom exceptions (OutOfStockError, InvalidCouponError, PaymentFailureError) inheriting from Exception.
60. File I/O: append new orders to orders_log.txt; read product catalogue from products.csv; write invoices to invoices/ directory.
61. OOP hierarchy: Entity > Product > PhysicalProduct / DigitalProduct; abstract method calculate_shipping().
62. Polymorphism: apply_discount() behaves differently for Premium vs Guest customers.
63. Operator overloading: Cart + Cart merges two carts; Product * int replicates order quantity; __str__ for pretty print.
64. Functions: place_order(user_id, cart, *, coupon=None, gift_wrap=False), search_products(*categories, min_price=0, **filters).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (all arg types)
✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators

✔ match-case	✔ OOP: Inheritance
✔ Lists + List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets (frozenset)	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
grand_total = subtotal * (1 + tax_rate) - discount   # arithmetic
status |= 0b00000100   # set 'shipped' flag with bitwise OR
order_id_hex = f'ORD-{order_id:06X}'  # base conversion in f-string
sku = f"{cat[:3].upper()}-{pid:05d}"  # string methods
```

Expected Input / Output

Input: products.csv + interactive cart CLI. Output: invoice_{order_id}.txt + orders_log.txt.

Problem 4: Hospital Patient Record System

Difficulty	Intermediate	Domain	Healthcare / Data Management

Objective

Create a secure patient record management system for a clinic, covering admissions, prescriptions, billing, and discharge summaries.

Problem Description

Design a hospital management system that handles patient registrations, doctor assignments, prescription management, billing, and report generation. Emphasize exception safety and data integrity throughout.

Tasks to Implement

65. Patient records stored as dictionaries with keys: id, name, age, blood_group, diagnoses (list), prescriptions (list of dicts), bill_items (list of tuples).
66. Maintain ward-wise patient lists. Apply all list methods; use list comprehension to find patients with a specific diagnosis.
67. Store blood_group as string; available_blood_groups as a frozenset; donation_pool as a set with add/discard/update.
68. Use a tuple to represent an immutable patient admission record (id, name, date, ward) for archival.
69. Nested dictionaries for doctor schedules: {dept: {doctor_name: [time_slots]}}. Use all dict methods.
70. Generator patient_stream(ward) yields patient records without loading all wards into memory.
71. AppointmentIterator class yields upcoming appointments in sorted order using __iter__/__next__.
72. Arithmetic: compute BMI (weight/height**2), bill total with GST, insurance coverage, out-of-pocket amount.
73. Bitwise: encode patient alert flags (allergy=1, isolation=2, critical=4, VIP=8) in a single int field.
74. Logical/comparison: validate prescription dosage, check bed availability, verify doctor credentials.
75. if/elif/else for triage severity; match-case for ward routing based on diagnosis category.
76. Base conversions: patient ID in hex on wristband, blood pressure reading display in different formats, bed number in binary.
77. String methods: parse doctor notes with split/strip/replace; format prescription labels with upper/title/center/format; validate email with endswith/find.

78. for with enumerate for prescription numbering; nested for for ward-bed matrix; while for admission form retry loop.
79. break when a matching patient is found in search; continue to skip discharged patients; pass as stub for future payment gateway.
80. try/except for age/weight type conversion; FileNotFoundError for missing record files; custom PatientNotFoundError.
81. File I/O: read/write patient records to patients.json-style text file; append to audit_log.txt; read config from hospital.cfg.
82. OOP: Person > Patient, Doctor, Nurse. Abstract class HospitalEntity with abstract methods register() and generate_report().
83. Polymorphism: generate_report() produces different output for Patient vs Doctor vs Admin.
84. Operator overloading: PatientQueue + PatientQueue merges queues; __len__ returns queue size; __contains__ checks patient ID.
85. Functions: admit_patient(name, age, *, ward, doctor=None), prescribe(*medicines, dosage='twice daily', **meta).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (all arg types)
✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets (frozenset)	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
bmi = round(weight / height**2, 1)  # arithmetic
alert_flags |= 0b0100  # set 'critical' using bitwise OR
patient_id_hex = f'PAT-{pid:04X}'  # hex base conversion
label = f"{med.upper():=^30}"  # string center formatting
```

Expected Input / Output

Input: CLI menu for admissions, search, prescribe, discharge. Output: discharge_summary_{id}.txt.

Problem 5: Banking & Financial Transactions Simulator

Difficulty	Intermediate–Advanced	Domain	Finance / Banking

Objective

Simulate a core banking system with accounts, transactions, loans, and statements using comprehensive Python features.

Problem Description

Build a banking simulator that handles savings and current accounts, fixed deposits, fund transfers, loan EMI calculations, and monthly statement generation. Security, precision, and exception safety are paramount.

Tasks to Implement

86. Account objects with: account_no (int), holder (str), balance (float), transactions (list of tuples), account_type (str), pin_hash (int).
87. Ledger stored as a list of Account objects. List comprehension to filter accounts by type, minimum balance, or negative balance.
88. Transaction history as list of tuples: (timestamp_str, type, amount, balance_after). Demonstrate all tuple operations.
89. Maintain a set of blocked_account_numbers; set operations to find common blocked/flagged accounts.
90. Dictionary for interest_rates: {account_type: {tenure: rate}}. Exhaustive dict usage including dict comprehension.
91. Generator monthly_statement_generator(account) yields formatted lines for a statement without buffering all months.
92. TransactionIterator class that iterates transactions in reverse (most recent first) using __iter__/__next__.
93. Arithmetic: EMI = P*r*(1+r)^n / ((1+r)^n - 1); compound interest; penalty computation with modulo.
94. Bitwise: encode transaction flags (debit=1, credit=2, reversal=4, flagged=8, international=16).
95. Logical/comparison: overdraft protection, daily limit checks, balance sufficiency validation.
96. if/elif/else for transaction routing; match-case for account type operations.
97. Base conversions: account numbers in hex for IFSC, sort codes in octal, CVV hash display in binary.
98. String methods: parse bank statement CSV (split/strip/lstrip/rstrip); format currency with format()/f-string/zfill; validate IFSC with upper/isalpha/isdigit.
99. for with zip to match debits and credits in reconciliation; while for PIN retry (max 3); break on successful auth; continue to skip reversed transactions.
100. Type conversions: str to Decimal-like float, bool to int for flag encoding, int to hex/bin/oct for codes.
101. Custom exceptions: InsufficientFundsError, AccountFrozenError, DailyLimitExceededError, InvalidPINError.
102. File I/O: read opening balances from accounts.csv; append each transaction to ledger.txt; write statement to PDF-ready .txt file.
103. OOP: FinancialEntity > Account > SavingsAccount, CurrentAccount, FixedDeposit.
104. Abstract class Accountable with abstract methods deposit(), withdraw(), generate_statement().
105. Polymorphism: calculate_interest() uses simple interest for savings, compound for FD.
106. Operator overloading: Account + float to deposit; Account - float to withdraw; __str__ for mini-statement.
107. Functions: transfer(src, dst, amount, /, *, memo=''), calculate_emi(principal, rate, tenure, *, round_to=2).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (positional-only, keyword-only, mix)
✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance

✔ Lists + List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
emi = (p * r * (1+r)**n) / ((1+r)**n - 1)  # arithmetic with **
txn_flag |= 0b00010  # credit flag via bitwise OR
ifsc = f'{bank_code.upper()}{branch:07d}'  # string + base conv
acc_no_hex = hex(account_no)[2:].upper().zfill(8)  # base conversion
```

Expected Input / Output

Input: accounts.csv + interactive banking CLI. Output: statement_{acc}.txt + ledger.txt.

Problem 6: Smart Inventory & Supply Chain Manager

Difficulty	Intermediate	Domain	Logistics / Inventory

Objective

Design an inventory management system for a warehouse that tracks stock, orders, suppliers, and generates reorder alerts.

Problem Description

Create a warehouse management system that maintains product stock levels, processes purchase and sales orders, computes reorder points, and interfaces with supplier records stored in text files.

Tasks to Implement

108. Product records: {sku: str, name: str, qty: int, unit_cost: float, reorder_level: int, supplier_id: int, tags: list}.
109. Inventory stored as a list of dicts. List comprehension for low-stock filter; nested comprehension for category-supplier matrix.
110. Use a tuple to store fixed warehouse dimensions (aisles, rows, columns) for slot calculation.
111. Set of active_sku_codes; set operations to find discontinued, new, and common products across two warehouses.
112. Supplier directory as nested dict: {supplier_id: {name, contact, lead_time, products: list}}.
113. Generator low_stock_alert_generator() that yields (sku, qty, reorder_level) only for items below threshold.
114. InventoryIterator that walks aisles in row-major order via __iter__/__next__.
115. Arithmetic: reorder_qty = (avg_daily_sales * lead_time) + safety_stock - current_stock; holding cost % computation.
116. Bitwise: warehouse zone flags (refrigerated=1, flammable=2, fragile=4, oversize=8, hazmat=16).
117. Logical/comparison: validate order quantities, check supplier credit limits, verify expiry dates.
118. if/elif/else for order priority; match-case for product category routing to correct aisle.
119. Base conversions: SKU codes contain hex product ID; bin to represent storage flags; oct for legacy shelf codes.
120. Type conversions: CSV parsing (str to int/float); bool flags to int for bitfields; explicit int() for qty truncation.
121. String methods: generate SKU with upper/replace/zfill/format; parse supplier CSV with split/strip; search with find/in/startswith.
122. for with enumerate to assign slot numbers; while for user input retry; break when item found; continue to skip expired items; pass for future RFID stub.

123. try/except: KeyError for missing SKU, ValueError for bad quantities, FileNotFoundError for missing files.
124. File I/O: read inventory.csv; write reorder_report.txt; append receipt.txt for each shipment received.
125. OOP: WarehouseEntity > Product > PerishableProduct, DigitalGood; abstract method check_expiry().
126. Polymorphism: calculate_storage_cost() differs for perishable vs non-perishable.
127. Operator overloading: Inventory + Inventory merges two warehouses; __getitem__ for SKU-based lookup; __len__ for total SKUs.
128. Functions: receive_shipment(supplier_id, *items, warehouse='main', **metadata), reorder(sku, /, qty=None).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (all arg types)
✔ Arithmetic Operators	✔ Exception Handling
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + Nested List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
reorder_qty = (avg_sales * lead_time) + safety - current_qty  # arithmetic
zone_flag = 0; zone_flag |= 0b00100 if fragile else zone_flag  # bitwise
sku = f"{cat[:2].upper()}{pid:05X}"  # hex in SKU
name_parts = raw.strip().split(',')  # string parsing
```

Expected Input / Output

Input: inventory.csv + purchase_orders.txt. Output: reorder_report.txt + receipt_{date}.txt.

Problem 7: Social Media Analytics Dashboard

Difficulty	Intermediate	Domain	Social Media / Analytics

Objective

Parse and analyse social media post data to extract trends, engagement metrics, and user behaviour patterns.

Problem Description

Build a text-based analytics engine that processes post data (stored in files), computes engagement metrics, identifies trending hashtags, flags inappropriate content, and generates daily summary reports.

Tasks to Implement

129. Post record: {post_id: int, user: str, content: str, likes: int, shares: int, comments: int, timestamp: str, hashtags: list, flags: int}.
130. Posts stored in a list; list comprehension to filter by date range, minimum likes, or specific hashtag.
131. Use tuples for immutable platform config: (max_chars, max_hashtags, max_daily_posts).
132. Set of trending_hashtags; set operations to compare hashtag overlap between two users.
133. Dictionary: {user: {posts: int, total_likes: int, followers: int, following: set}}; full dict usage.
134. Generator trending_posts_generator(threshold) yields posts with engagement above threshold lazily.
135. HashtagIterator yields hashtags in descending frequency order via __iter__/__next__.
136. Arithmetic: engagement_rate = (likes + shares + comments) / reach * 100; weighted trending score.
137. Bitwise: content flags (spam=1, nsfw=2, hate_speech=4, misinformation=8, verified=16).
138. Logical/comparison: content moderation rules, rate-limit checks, follower threshold for verification.
139. if/elif/else for content moderation; match-case for notification type routing.
140. Base conversions: post IDs in hex for URLs, user IDs in binary for privacy mask, shard keys in octal.
141. Type conversion: parse string timestamps; convert like counts from string to int; bool to int for flag encoding.
142. String methods: extract hashtags with split/strip/startswith; sentiment keyword search with find/count/in; format report with center/ljust/rjust/format.
143. for with enumerate to rank top posts; nested for to build user-hashtag matrix; while for paginated data fetch simulation.
144. break when banned word found; continue to skip deleted posts; pass as stub for ML sentiment model.
145. try/except: ValueError for engagement calculation on zero-reach posts, KeyError for missing user fields.
146. File I/O: read posts.txt; write daily_report_{date}.txt; append moderation_log.txt per flagged post.
147. OOP: ContentEntity > Post, Story, Reel; abstract method compute_reach().
148. Polymorphism: generate_preview() produces text/image/video previews for different content types.
149. Operator overloading: PostFeed + PostFeed merges feeds; __contains__ checks if a post_id exists; __len__ returns feed size.
150. Functions: analyse_user(user, /, metric='engagement', *, start_date, end_date), filter_posts(*hashtags, min_likes=0, **criteria).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (all arg types)
✔ Arithmetic Operators	✔ Exception Handling
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
er = (likes + shares + comments) / max(reach, 1) * 100  # arithmetic
flags |= 0b0010 if nsfw_detected else flags  # bitwise flag setting
```

```
post_url = f'https://site.com/p/{post_id:08X}'  # hex base conversion
tags = [w for w in content.split() if w.startswith('#')]  # list comp + string
```

Expected Input / Output

Input: posts.txt (one JSON-like record per line). Output: daily_report_{date}.txt + moderation_log.txt.

Problem 8: Airline Flight Reservation System

Difficulty	Advanced	Domain	Travel / Reservation

Objective

Build a full-featured airline reservation system covering flight search, seat allocation, ticketing, check-in, and boarding pass generation.

Problem Description

Simulate an airline reservation system that manages flights, passengers, bookings, dynamic pricing, seat maps, and boarding operations. The system must be robust against invalid inputs and persist all bookings to files.

Tasks to Implement

151. Flight: {flight_no: str, origin: str, dest: str, dep_time: str, seats: list, fare: float, status: int}.
152. Flights stored in a list; list comprehension for route search; nested comprehension for seat-class map.
153. Seat map as a tuple of tuples (rows × cols); use tuple indexing to locate specific seats.
154. Set of checked_in_passengers; set operations to find no-shows (booked - checked_in).
155. Booking dictionary: {pnr: {passenger, flight_no, seat, fare_paid, extras: dict}}; all dict operations.
156. Generator available_seats_generator(flight) lazily yields available seat labels.
157. BoardingIterator yields passengers in boarding zone order via __iter__/__next__.
158. Arithmetic: dynamic_fare = base_fare * load_factor_multiplier + surcharge - discount; baggage fee calculation.
159. Bitwise: seat feature flags (window=1, aisle=2, extra_legroom=4, near_exit=8, bulkhead=16).
160. Logical/comparison: validate passport expiry, check baggage weight limits, verify ticket class eligibility.
161. if/elif/else for check-in rules; match-case for booking status (CONFIRMED, WAITLISTED, CANCELLED, CHECKED_IN, BOARDED).
162. Base conversions: PNR includes hex flight ID; seat row in octal for gate display; boarding sequence in binary.
163. Type conversions: parse fare string to float; seat row str to int; bool upgrade_eligible to int for fare modifier.
164. String methods: generate PNR with upper/replace/zfill; parse route codes with split/upper; format boarding pass with center/ljust/format.
165. for with enumerate to assign boarding sequence; while for payment retry; break on successful booking; continue to skip cancelled flights; pass for future loyalty module.
166. Custom exceptions: SeatUnavailableError, FlightFullError, CheckInClosedError, InvalidPNRError.
167. File I/O: read flight_schedule.txt; write booking_confirmation_{pnr}.txt; append bookings.csv for each new booking.
168. OOP: TravelEntity > Flight, Passenger, Booking; abstract class Bookable with abstract methods book() and cancel().
169. Polymorphism: calculate_fare() differs for EconomyFlight, BusinessFlight, CharterFlight.
170. Operator overloading: FlightSchedule + FlightSchedule merges schedules; __contains__ checks PNR in booking system; __str__ prints flight card.
171. Functions: book_seat(pnr, flight_no, seat, /, *, meal_pref='none', extra_baggage=0), search_flights(origin, dest, *, date, cabin='economy').

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass

✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (positional-only, keyword-only, mix)
✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + Nested List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
fare = base * load_factor + surcharge - (base * disc_pct / 100)  # arithmetic
seat_flags = 0b00001 | 0b01000  # window + near_exit
pnr = f"{flight_no.upper()}{booking_id:04X}"  # hex in PNR
boarding_pass = f"{'BOARDING PASS':=^50}"  # string formatting
```

Expected Input / Output

Input: flight_schedule.txt + interactive booking CLI. Output: booking_{pnr}.txt + bookings.csv.

Problem 9: Smart Home Automation Controller

Difficulty	Intermediate	Domain	IoT / Home Automation

Objective

Simulate a smart home controller that manages devices, schedules, energy consumption, automation rules, and event logging.

Problem Description

Build a smart home automation system that manages IoT devices (lights, AC, security cameras, smart plugs), schedules automation rules, monitors energy consumption, handles device errors gracefully, and logs all events to files.

Tasks to Implement

172. Device record: {device_id: int, name: str, type: str, state: int, power_w: float, room: str, schedule: list, error_count: int}.
173. All devices stored in a list; list comprehension to find active devices, devices by room, or high-power devices.
174. Use a tuple for immutable room layout: (('Living Room', 'Bedroom'), ('Kitchen', 'Bathroom'), ('Garage',)).
175. Set of online_device_ids; set operations to find offline, newly-connected, or conflicting devices.
176. Automation rules stored as nested dict: {rule_id: {trigger, action, devices: list, active: bool}}.
177. Generator energy_report_generator(devices) yields (name, power_w, daily_kwh) for active devices.
178. DeviceIterator that iterates devices grouped by room via __iter__/__next__.
179. Arithmetic: energy_cost = power_w / 1000 * hours * tariff_per_kwh; efficiency ratio; monthly bill projection.

180. Bitwise: device state encoding (on=1, standby=2, error=4, locked=8, scheduled=16, overridden=32).
181. Logical/comparison: validate schedule times, check energy budget thresholds, verify room occupancy sensors.
182. if/elif/else for device command routing; match-case for device type (light/ac/security/appliance).
183. Base conversions: device IDs in hex for MAC-like address display; state bitmask in binary for debug; room codes in octal.
184. Type conversions: parse schedule time strings to int hours/minutes; bool sensor readings to int; float power to int for integer arithmetic.
185. String methods: format device name with title/upper; parse command strings with split/strip/lower; build MAC address with zfill/join/upper.
186. for with enumerate to number automation rules; while for main event loop; break on emergency shutdown; continue to skip offline devices; pass for future voice-command stub.
187. Custom exceptions: DeviceOfflineError, ScheduleConflictError, EnergyBudgetExceededError.
188. File I/O: read device_config.json-style .txt; append event_log.txt for every state change; write energy_report_{date}.txt.
189. OOP: SmartDevice > SmartLight, SmartAC, SecurityCamera, SmartPlug; abstract method execute_command().
190. Polymorphism: get_status_report() returns different data structures for different device types.
191. Operator overloading: DeviceGroup + DeviceGroup merges rooms; __len__ returns device count; __contains__ for device_id lookup; __str__ for dashboard card.
192. Functions: set_schedule(device_id, /, *time_slots, mode='daily', **options), automate(*rules, trigger='time', priority=1).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (all arg types)
✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
cost = (power_w / 1000) * hours * tariff  # arithmetic
state |= 0b010000  # set 'scheduled' bit
mac = ':'.join(f'{b:02X}' for b in device_id.to_bytes(6, 'big'))  # hex + join
devices_on = [d for d in devices if d['state'] & 0b000001]  # bitwise in comp
```

Expected Input / Output

Input: device_config.txt + CLI commands. Output: event_log.txt + energy_report_{date}.txt.

Problem 10: Text-Based RPG Game Engine

Difficulty	Advanced	Domain	Game Development / Entertainment

Objective

Build a turn-based text RPG engine with character creation, combat, inventory, quests, save/load, and a world map — demonstrating every Python concept in a game context.

Problem Description

Create a complete text-based RPG (Role-Playing Game) engine with character classes, combat mechanics, an inventory system, spell casting, NPC interactions, quest tracking, and world map navigation. Every Python concept must appear naturally within the game logic.

Tasks to Implement

193. Character: {name: str, hp: int, max_hp: int, mp: int, atk: int, def_: int, level: int, xp: int, gold: float, inventory: list, spells: set, stats: dict, flags: int}.
194. World map stored as a list of lists (2D); list comprehension to find rooms with specific features or enemies.
195. Fixed dungeon layout as a tuple of tuples of strings representing room types; slice for sub-level extraction.
196. Set of discovered_rooms; set operations to find undiscovered areas (all_rooms - discovered).
197. Quest log as dict: {quest_id: {title, desc, objectives: list, completed: bool, reward: dict}}.
198. Generator loot_drop_generator(enemy_level) lazily yields items sampled from loot table.
199. CombatIterator that cycles through turn order (player, enemies sorted by speed) via __iter__/__next__.
200. Arithmetic: damage = (atk * weapon_multiplier) - (enemy_def * 0.5); XP to next level = base * level**1.5; gold drop with modulo for coin types.
201. Bitwise: character status effects (poisoned=1, burning=2, frozen=4, stunned=8, invisible=16, blessed=32).
202. Logical/comparison: check if spell can be cast (mp >= cost and level >= req and spell in spells).
203. if/elif/else for combat outcome; match-case for player command (attack/spell/item/flee/inspect).
204. Base conversions: save file uses hex checksums; map grid references in octal; item IDs in binary bit flags.
205. Type conversions: parse save file tokens (str to int/float); bool status flags to int for bitmask; chr()/ord() for cipher in treasure puzzle.
206. String methods: generate dungeon art with center/ljust; parse item descriptions with split/strip; combat log with format/f-string/upper/replace.
207. for with enumerate to number inventory items; while for main game loop; break on game-over; continue to skip dead enemies; pass for unimplemented room events.
208. Custom exceptions: InvalidCommandError, InsufficientMPError, QuestNotActiveError, SaveFileCorruptError.
209. File I/O: save game state to savegame.txt (structured text); load and parse; append combat_log.txt; read world_map.txt.
210. OOP: Entity > Character > Player, Enemy, Boss, NPC; abstract class Combatant with abstract methods attack() and take_damage().
211. Polymorphism: use_ability() behaves differently for Warrior (physical), Mage (magical), Rogue (stealth).
212. Operator overloading: Inventory + Inventory merges bags; Character += xp_amount to level up; __str__ for character sheet; __lt__ for initiative sorting.
213. Functions: cast_spell(caster, target, /, spell_name, *, empowered=False), equip(character, *items, slot=None, **enchantments).

Python Concepts Covered

✔ Variables & Literals	✔ for / while / break / continue / pass
✔ Datatypes & Type Conversion	✔ String Methods (all)
✔ Base Conversions	✔ Functions (positional-only, keyword-only, *args, **kwargs)

✔ Arithmetic Operators	✔ Custom Exceptions
✔ Bitwise Operators	✔ File Handling (save/load)
✔ Logical & Comparison Operators	✔ Iterators
✔ if / elif / else	✔ Generators
✔ match-case	✔ OOP: Inheritance
✔ Lists + 2D List Comprehension	✔ OOP: Polymorphism
✔ Tuples	✔ OOP: Operator Overloading
✔ Sets	✔ OOP: Abstraction
✔ Dictionaries	

Implementation Hints

```
dmg = int(atk * weapon_mult - enemy_def * 0.5)  # arithmetic + type conv
status |= 0b000001 if poisoned else status  # bitwise status flag
xp_needed = int(100 * level ** 1.5)  # exponent arithmetic
char_sheet = f"{'[ CHARACTER ]':=^40}"  # string formatting
```

Expected Input / Output

Input: interactive text commands + world_map.txt. Output: savegame.txt + combat_log.txt + console dungeon display.